R Notebook

Contents

Load packages and the dataset . . . 1
Data transformation . . . 2
Renaming and adjusting columns . . . 2
Combining control and experimental groups . . . 2
Dealing with multiplicity of measurements . . . 3
Data summarisation . . . 5
Converting summary data . . . 5
Subsetting by data type . . . 5
Converting Cohen's d to Hedge's g . . . 5
Comparing methods . . . 5
Assigning results . . . 5
Estimate Hedge's g from beta . . . 5
Estimation with 'esc' . . . 5
Estimation 1: Coefficient/Variance . . . 6
Estimation 2: MD/pooled SD . . . 6
Assigning results . . . 6
Calculating effect size from raw . . . 6
Using 'escalc' function in metafor . . . 6
Using 'esc' package . . . 7
Combine and save effect size data . . . 7
Meta-analysis . . . 7
Perform meta-analysis with 'metafor' . . . 7
Perform GOSH analysis with 'metafor' . . . 8
GOSH after removing outliers . . . 8
Perform meta-analysis with 'meta' . . . 8
Funnel plot . . . 8
Funnel plots (trim and fill) . . . 9
Funnel plots (trim and fill + outlier removed) . . . 9
Subgroup analysis by country . . . 9
Subgroup analysis by intervention type . . . 10

Load packages and the dataset

```
library(openxlsx)
library(metafor)
library(esc)
library(dmetar)
library(metafor)
library(dplyr)
```

There are two well-maintained R packages available for meta-analysis, namely 'metafor' and 'meta'. Here we start with 'metafor' as it provides a detailed example about how to perform meta-analysis for the pre-post study design with continuous variables. As usual, we start with loading the dataset, but we will divide the

dataset into two subsets. One for the control group and the other for the experimental group. We further restrict the columns we used in the dataset for simplification.

```
Dataset <- read.xlsx('~/Documents/Coding/HP407/Dataset final (RoB mod).xlsx', 'Combined')
# colnames(Dataset)
Dataset <- Dataset[c('Study.Identifier', 'Country',
                     'Caregiver.intervention.type',
                     'Caregiver.intervention.subgroups',
                     'Effect.measure','Measurement.method',
                     'Measurement.type',
                     'Pre_mean','Pre_N','Pre_SD',
                     'Post_mean','Post_N','Post_SD',
                     'Duration',
                     'Measurement.points.(days.from.start)',
                     'Treatment.duration.(days.from.start)',
                     'Overall')]
```

Data transformation

Renaming and adjusting columns

```
colnames(Dataset)[which(colnames(Dataset)=="Measurement.points.(days.from.start)")] <-
  "Measurement.point"
Dataset$Measurement.point <- as.numeric(Dataset$Measurement.point)
colnames(Dataset)[which(colnames(Dataset)=="Treatment.duration.(days.from.start)")] <-
  "Treatment.duration"
Dataset$Country <-
  relevel(as.factor(Dataset$Country), "Vietnam")
Dataset$Caregiver.intervention.type <-
  as.factor(Dataset$Caregiver.intervention.type)
Dataset$Caregiver.intervention.type <-
  relevel(Dataset$Caregiver.intervention.type, 'No intervention')
```

Combining control and experimental groups

Then, we convert all the data from control and experimental groups of the same study and measurement to the same row with the following code:

```
data_cols <- c("Pre_mean","Pre_N","Pre_SD","Post_mean","Post_N","Post_SD")
info_cols <- colnames(Dataset)[-which(colnames(Dataset) %in% data_cols)]
meta_df.ctrl <-
  Dataset[Dataset$Caregiver.intervention.type == 'No intervention', ]
meta_df.expt <-
  Dataset[Dataset$Caregiver.intervention.type != 'No intervention',]
meta_df.es <- list()
for (i in 1:nrow(meta_df.expt)){
    item <- meta_df.expt[i, ]
    item_info <- item[info_cols]
    item_data <- item[data_cols]
    colnames(item_data) <- c("Expt_Pre_mean","Expt_Pre_N","Expt_Pre_SD",
                             "Expt_Post_mean","Expt_Post_N","Expt_Post_SD")

    # selection by study
    item_data_ctrl <-
      meta_df.ctrl[meta_df.ctrl$Study.Identifier == item_info$Study.Identifier, ]
```

```
    # selection by effect measure
    item_data_ctrl <-
      item_data_ctrl[item_data_ctrl$Effect.measure  == item_info$Effect.measure, ]

    # selection by measurement method
    item_data_ctrl <-
      item_data_ctrl[item_data_ctrl$Measurement.method  == item_info$Measurement.method, ]

    # selection by measurement type
    item_data_ctrl <-
      item_data_ctrl[item_data_ctrl$Measurement.type    == item_info$Measurement.type, ]

    # selection by intervention duration
    item_data_ctrl <-
      item_data_ctrl[item_data_ctrl$Duration    == item_info$Duration, ]

    # selection by measurement point
    item_data_ctrl <-
      item_data_ctrl[item_data_ctrl$Measurement.point   ==
                       item_info$Measurement.point, ]

    # remove duplicates
    item_data_ctrl <- unique(item_data_ctrl)

    # check whether the selection is unique
    if (nrow(item_data_ctrl) > 1){
      print(item$Study.Identifier)
      print(item$Effect.measure)
      print(item_data_ctrl)
    }

    # subset data from the selection
    item_data_ctrl <- item_data_ctrl[data_cols]

    # rename the columns of selected data
    colnames(item_data_ctrl) <- c("Ctrl_Pre_mean","Ctrl_Pre_N","Ctrl_Pre_SD",
                                  "Ctrl_Post_mean","Ctrl_Post_N","Ctrl_Post_SD")

    # combine the data into a data entry
    data_item <- cbind(item_info, item_data, item_data_ctrl)

    # combine the data entry with previous records
    meta_df.es  <- rbind(meta_df.es , data_item)

}
```

When we finish the transformation, we perform multiplicity detection to make sure each measurement is represented by only one data entry, in case of multiple eligible measurement points and scales.

Dealing with multiplicity of measurements

```
# multiplicity detection
temp_list <- list()
```

```
for (effect.measure in unique(meta_df.es$Effect.measure)){
    temp <- meta_df.es[meta_df.es$Effect.measure == effect.measure, ]
    temp_data <- data.frame(table(temp$Study.Identifier))
    temp_list <- c(temp_list, as.character(temp_data[which(temp_data$Freq > 1), ]$Var1))
}
print(unique(temp_list))
```

Thus, we are able to know the listed studies involve multiplicity of measurement. Thus, we need to further adjust the dataset study by study.

```
# 'Mahdavi 2017' has two intervention groups, so there is no need to change.
# 'Pahlavanzadeh 2010' has two measurement points, we choose 35 days at the end of intervention
meta_df.es <- meta_df.es[- which(meta_df.es$Study.Identifier == 'Pahlavanzadeh 2010' & meta_df.es$'Measu
print(dim(meta_df.es))

# 'Shata 2017' has two measurement points, we choose 56 days at the end of intervention
meta_df.es <-
  meta_df.es[- which(meta_df.es$Study.Identifier == 'Shata 2017' &
                       meta_df.es$'Measurement.point' == 56), ]
print(dim(meta_df.es))

# 'Wang 2014a' has two measurement points, we choose 365 days at the end of intervention
meta_df.es <-
  meta_df.es[- which(meta_df.es$Study.Identifier == 'Wang 2014a' &
                       meta_df.es$'Measurement.point' == 180), ]
print(dim(meta_df.es))

# 'Zarepour 2020' has two measurement points, we choose 58 days, one month after the end of interventio
meta_df.es <- meta_df.es[- which(meta_df.es$Study.Identifier == 'Zarepour 2020' &
                                   meta_df.es$'Measurement.point' == 118), ]
print(dim(meta_df.es))

# 'Pan 2019' has two measurement points, we choose 150 days, one month after the end of intervention al
meta_df.es <- meta_df.es[- which(meta_df.es$Study.Identifier == 'Pan 2019' &
                                   meta_df.es$'Measurement.point' == 210), ]
print(dim(meta_df.es))

# 'Guerra 2011' has two measurement scales for caregiver neuropsychiatric symptoms, we choose NPI-Q sev
meta_df.es <- meta_df.es[- which(meta_df.es$Study.Identifier == 'Guerra 2011' & meta_df.es$Measurement.
print(dim(meta_df.es))

# 'Orawan 2018' has three measurement points, we choose 56 days, the closest measurement point to the e
meta_df.es <- meta_df.es[- which(meta_df.es$Study.Identifier == 'Orawan 2018' & meta_df.es$'Measurement
print(dim(meta_df.es))

# multiplicity detection again!
temp_list <- list()
for (effect.measure in unique(meta_df.es$Effect.measure)){
    temp <- meta_df.es[meta_df.es$Effect.measure == effect.measure, ]
    temp_data <- data.frame(table(temp$Study.Identifier))
    temp_list <- c(temp_list, as.character(temp_data[which(temp_data$Freq > 1), ]$Var1))
}
unique(temp_list)
```

```
# meta_df.es[which(meta_df.es$Study.Identifier == 'Guerra 2011'), ]
```

Data summarisation

```
es.stats <- aggregate(meta_df.es[c('Expt_Post_N', 'Ctrl_Post_N')], list(meta_df.es$Effect.measure), FUN=
es.stats.freq <- data.frame(table(meta_df.es$Effect.measure))
colnames(es.stats)[1] <- 'Effect.measure'
es.stats <- bind_cols(es.stats, 'Freq'=es.stats.freq$Freq)
es.stats$Sample_size <- es.stats$Expt_Post_N + es.stats$Ctrl_Post_N
es.stats
```

Converting summary data

Subsetting by data type

```
es_df <- meta_df.es[meta_df.es$Effect.measure == 'Caregiver burden', ]
es_df.raw <- es_df[es_df$Measurement.type == 'Raw',]
es_df.regression <- es_df[es_df$Measurement.type == 'Regression',]
es_df.smd <- es_df[es_df$Measurement.type == 'SMD',]
```

Converting Cohen's d to Hedge's g

```
# two different methods to calculate Hedge's g should get the same result
es_df.smd$Expt_Post_mean * (1- (3)/(4*(es_df.smd$Expt_Post_N + es_df.smd$Ctrl_Post_N) - 9))
hedges_g(es_df.smd$Expt_Post_mean, es_df.smd$Expt_Post_N + es_df.smd$Ctrl_Post_N)
```

Comparing methods

```
# thus, we assign the values to the effect size columns
es_df.smd$mean.effect <-
  hedges_g(es_df.smd$Expt_Post_mean, es_df.smd$Expt_Post_N + es_df.smd$Ctrl_Post_N)
es_df.smd$variance <- es_df.smd$Expt_Post_SD^2
es_df.smd$standard_error <-
  es_df.smd$Expt_Post_SD/sqrt( es_df.smd$Expt_Post_N + es_df.smd$Ctrl_Post_N)
es_df.smd
```

Assigning results

Estimate Hedge's g from beta

```
# esc outputs
esc_B(es_df.regression$Expt_Post_mean, es_df.regression$Expt_Post_SD,
      es_df.regression$Expt_Post_N, es_df.regression$Ctrl_Post_N, es.type = 'd')
esc_B(es_df.regression$Expt_Post_mean, es_df.regression$Expt_Post_SD,
      es_df.regression$Expt_Post_N, es_df.regression$Ctrl_Post_N, es.type = 'g')
```

Estimation with 'esc'

```
# Estimation of ES from beta: Coefficient/Variance
print("Cohen's d:")
```

```
es_df.regression$Expt_Post_mean/es_df.regression$Expt_Post_SD
print("Hedge's g:")
es_df.regression$Expt_Post_mean/es_df.regression$Expt_Post_SD *
  (1- (3)/(4*(es_df.smd$Expt_Post_N + es_df.smd$Ctrl_Post_N) - 9))
```

Estimation 1: Coefficient/Variance

```
# Estimation of ES from beta: pooled variance
# Calculate d according to https://mason.gmu.edu/~dwilsonb/downloads/esformulas.pdf
# But the result is very similar to 'esc' output
s_y <- es_df.regression$Expt_Post_SD
N <- es_df.regression$Expt_Post_N + es_df.regression$Ctrl_Post_N
n1 <- es_df.regression$Ctrl_Post_N
n2 <- es_df.regression$Expt_Post_N
b <- es_df.regression$Expt_Post_mean
print("Cohen's d:")
s_pooled <- sqrt(((s_y^2)*(N-1)-(b^2)*(n1*n2/(n1+n2)))/(N-2))
b/s_pooled
print("Hedge's g:")
(b/s_pooled)* (1- (3)/(4*(es_df.regression$Expt_Post_N + es_df.regression$Ctrl_Post_N) - 9))
```

Estimation 2: MD/pooled SD

```
es_df.regression.SMD <- data.frame(esc_B(es_df.regression$Expt_Post_mean,
                                         es_df.regression$Expt_Post_SD,
                                         es_df.regression$Expt_Post_N,
                                         es_df.regression$Ctrl_Post_N, es.type = 'g',
                                         study = es_df.regression$Study.Identifier))
es_df.regression$mean.effect <- es_df.regression.SMD$es
es_df.regression$variance <- es_df.regression.SMD$var
es_df.regression$standard_error <- es_df.regression.SMD$se
es_df.regression
```

Assigning results

Calculating effect size from raw

```
es_df.transformed <- escalc(m1i=Expt_Post_mean, sd1i=Expt_Post_SD, n1i=Expt_Post_N,
                            m2i=Ctrl_Post_mean, sd2i=Ctrl_Post_SD, n2i=Ctrl_Post_N,
                            measure="SMD", vtype="UB", data=es_df.raw,
                            slab = Study.Identifier, var.names = c('mean.effect', 'variance'))
#es_df.transformed$mean.effect
#es_df.transformed$variance
```

Using 'escalc' function in metafor

```
reserved_cols <- c('Ctrl_Pre_mean', 'Ctrl_Pre_SD', 'Ctrl_Post_mean', 'Ctrl_Post_SD',
                   'Ctrl_Post_N',
                   'Expt_Pre_mean', 'Expt_Pre_SD', 'Expt_Post_mean', 'Expt_Post_SD',
                   'Expt_Post_N',
```

```
                    'Study.Identifier')
es_df.raw.data <- es_df.raw[reserved_cols]

row_SMD<- function(row_data){
  transformed <- esc_mean_gain(

    pre1mean = as.numeric(row_data[1]),
    pre1sd = as.numeric(row_data[2]),
    post1mean = as.numeric(row_data[3]),
    post1sd = as.numeric(row_data[4]),
    grp1n = as.numeric(row_data[5]),

    pre2mean = as.numeric(row_data[6]),
    pre2sd = as.numeric(row_data[7]),
    post2mean = as.numeric(row_data[8]),
    post2sd = as.numeric(row_data[9]),
    grp2n = as.numeric(row_data[10]),

    es.type = 'g',
    study = row_data[11])
  df <- data.frame(transformed)
  rownames(df) <- NULL
  return(df)
}
es_df.raw.SMD <- do.call('rbind', apply(es_df.raw.data, 1, row_SMD))
es_df.raw$mean.effect <- es_df.raw.SMD$es
es_df.raw$variance <- es_df.raw.SMD$var
es_df.raw$standard_error <- es_df.raw.SMD$se
```

Using 'esc' package

Combine and save effect size data

```
colnames(es_df.smd) <- colnames(es_df.raw)
colnames(es_df.regression) <- colnames(es_df.raw)
es_df.g <- rbind(es_df.raw, es_df.smd,es_df.regression)
write.csv(es_df.g, 'caregiver_burden_subset.csv')
```

Meta-analysis

Perform meta-analysis with 'metafor'

```
library(metafor)
res <- rma(mean.effect, variance, data=es_df.g, slab = Study.Identifier)
res
forest(res)
```

```
gosh_diac <- gosh.diagnostics(gosh(res))
plot(gosh_diac, which = "cluster")
plot(gosh_diac, which = "outlier")
gosh_diac
```

Perform GOSH analysis with 'metafor'

```
res <- rma(mean.effect, variance,
           data=es_df.g[-1, ],
           slab = Study.Identifier)
gosh_diac <- gosh.diagnostics(gosh(res))
plot(gosh_diac, which = "cluster")
plot(gosh_diac, which = "outlier")
gosh_diac
```

GOSH after removing outliers

Perform meta-analysis with 'meta'

```
library(meta)
pdf('caregiver_burden_forest.pdf', width = 10, height = 7)
m.gen <- metagen(TE = mean.effect,
                 seTE = standard_error,
                 studlab = Study.Identifier,
                 data = es_df.g,
                 sm = "SMD",
                 comb.fixed = FALSE)
forest(m.gen, prediction = TRUE)
dev.off()
forest(m.gen, prediction = TRUE)
```

```
pdf('caregiver_burden_funnel.pdf', width = 10, height = 7)
# funnel(m.gen, studlab = TRUE, xlim = c(-8, 1))
col.contour = c("gray75", "gray85", "gray95")
funnel.meta(m.gen, xlim = c(-6.2, 1.2),
            contour = c(0.9, 0.95, 0.99),
            col.contour = col.contour,
            studlab = TRUE, pos.studlab = '4')

# Add a legend
legend(x = -4, y = 0.01,
       legend = c("p < 0.1", "p < 0.05", "p < 0.01"),
       fill = col.contour)
dev.off()
```

Funnel plot

```
# Using all studies
tf <- trimfill(m.gen)

# Analyze with outliers removed
tf.no.out <- trimfill(update(m.gen,
                             subset = -c(3, 16)))

# Define fill colors for contour
contour <- c(0.9, 0.95, 0.99)
```

```
col.contour <- c("gray75", "gray85", "gray95")
ld <- c("p < 0.1", "p < 0.05", "p < 0.01")

# Use 'par' to create two plots in one row (row, columns)
par(mfrow=c(1,2))

pdf('caregiver_burden_funnel_TAF.pdf', width = 10, height = 7)
# Contour-enhanced funnel plot (full data)
funnel.meta(tf,
            xlim = c(-6.2, 1.2), contour = contour,
            col.contour = col.contour,
            studlab = TRUE, pos.studlab = '4')
legend(x = -4, y = 0.01,
       legend = ld, fill = col.contour)
title("Funnel Plot (Trim & Fill Method)")
dev.off()

pdf('caregiver_burden_funnel_Outlier.pdf', width = 10, height = 7)
# Contour-enhanced funnel plot (outliers removed)
funnel.meta(tf.no.out,
            xlim = c(-6.2, 1.2), contour = contour,
            col.contour = col.contour,
            studlab = TRUE, pos.studlab = '4')
legend(x = -4, y = 0.01,
       legend = ld, fill = col.contour)
title("Funnel Plot (Trim & Fill Method) - Outliers Removed")
dev.off()
```

Funnel plots (trim and fill)

```
pdf('caregiver_burden_forest_drop_outlier.pdf', width = 10, height = 7)
m.gen <- metagen(TE = mean.effect,
                 seTE = standard_error,
                 studlab = Study.Identifier,
                 data = es_df.g[-which(es_df.g$Study.Identifier == 'Govindakumari 2020'), ],
                 sm = "SMD",
                 comb.fixed = FALSE)
forest(m.gen, prediction = TRUE)
dev.off()
```

Funnel plots (trim and fill + outlier removed)

```
caregiver_burden_Country <- update.meta(m.gen,
            subgroup = Country,
            tau.common = FALSE)
caregiver_burden_Country
```

Subgroup analysis by country

```
caregiver_burden_Intervention <- update.meta(m.gen,
            subgroup = Caregiver.intervention.type,
```

```
            tau.common = FALSE)
pdf('caregiver_burden_subgroup.pdf', width = 15, height = 10)
forest(caregiver_burden_Intervention, subgroup.name = NULL, prediction = TRUE, prediction.subgroup = TRU
dev.off()
```

```
es_df.g[-1, ]
```

Subgroup analysis by intervention type